

# Should User Evidence Stay in Context or Enter Model Weights?

A Design Note on Personalized Models

August 29, 2026

## 1 Decision

New user preferences should first stay in a prompt or external memory. This makes them easy to inspect, change, and delete.

Train a user adapter only when a preference:

- appears in several sessions;
- remains stable over time;
- affects several kinds of tasks; and
- should apply even when the prompt does not include a user profile.

A parameter update is not the default choice. It must personalize better than context without reducing task success.

This note proposes a simple test. Use the same user evidence to build two versions:

1. **Context version:** Store the evidence in the prompt or external memory. Do not change the model.
2. **Parameter version:** Train a user adapter on the same evidence. Do not include that evidence in the test prompt.

Both versions start from the same model and run on the same later tasks. Choose the parameter version only when it improves personalization and preserves task success. Otherwise, keep the context version.

This is a proposed experiment. It is not yet a validated algorithm.

## 2 Three Forms of User Information

Table 1: Each form of user information serves a different purpose.

| Form                          | Use                                                                | Main property                                                                    |
|-------------------------------|--------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Prompt or external memory     | Provides user facts, preferences, or history at runtime.           | Easy to change, but uses context and may retrieve the wrong record.              |
| Training prompt               | Gives a teacher model instructions for creating a training target. | Provides training signal. It is not necessarily part of the deployed user state. |
| User adapter $\Delta\theta_u$ | Stores parameters learned for user $u$ .                           | Works without repeated profile text, but is harder to inspect and change.        |

In this note, an online update happens after deployment. The system first serves the user. It then learns from new interactions and creates a new adapter. Training on a fixed dataset is not post-deployment learning, even when the training process draws new samples.

A personalized agent must also complete tasks. Changing writing style alone does not make a system a personalized agent.

### 3 Sources for the Design

The current project code may change or contain errors. It is not evidence for the method. The design below comes from published papers.

Table 2: Source of each design choice.

| Choice                                  | Source                                                                                                                                           | Use in this note       |
|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| Split records by time                   | SPRInG divides each user’s records into five time periods. It uses the first 90% of each period for updates and the last 10% for evaluation [3]. | Adopt                  |
| Compare context and parameters          | OPPU compares retrieval, profile prompting, and per-user PEFT [1]. SPRInG also evaluates adapter and retrieval components.                       | Required baseline      |
| Compare external and learned experience | EvolveR keeps experience in context in its main method. It also tests writing that experience into parameters [4].                               | Motivation             |
| Measure task success                    | EvolveR uses final-answer correctness as an outcome reward. Our agent must also finish its task.                                                 | Separate metric        |
| Safety checks                           | SPRInG lists safety filters and poisoning defenses as future work.                                                                               | Not a core method step |
| Confidence intervals                    | SPRInG uses confidence intervals to report results. It does not use them to decide each update.                                                  | Reporting only         |
| Rollback                                | The closest papers do not provide a rollback method that we can reuse.                                                                           | Omit for now           |

### 4 What Prior Work Covers

**OPPU.** OPPU trains one parameter-efficient fine-tuning (PEFT) module from each user’s history [1]. It can also use retrieval or a user profile. OPPU is an offline method. It does not update from later deployed interactions.

**Personalized DPO.** Personalized RLHF learns user representations and LoRA parameters from offline preference data [2]. It does not study external memory or task outcomes from tool-using agents.

**SPRInG.** SPRInG is the closest personalization method [3]. It uses a user LoRA adapter and a retrieval buffer, then updates both over time. Therefore, combining external memory with an adapter is not a new contribution.

SPRInG selects training records by their change in likelihood. Records that remain hard after training enter its buffer. It does not use the same evidence to build matched context and parameter versions, then select one on later tasks.

**EvolveR.** EvolveR summarizes lessons from agent trajectories and stores them in an external Experience Base [4]. It also tests learning these lessons in model parameters. The reported average score falls from 0.382 to 0.371. This result suggests that writing unfiltered experience into parameters can add noise.

**CoSteer.** CoSteer does not train user parameters [5]. A local model runs with and without private user context. The difference between those outputs steers a cloud model. CoSteer shows why a runtime method should remain a baseline.

## 5 Proposed Test

At time  $t$ , user  $u$  has adapter  $\Delta\theta_u^{(t)}$  and external memory  $M_u^{(t)}$ . Let  $E_t$  be new evidence from later interactions.

**Context version.**

$$M_{u,\text{ctx}} = \text{MemoryUpdate}(M_u^{(t)}, E_t), \quad \Delta\theta_{u,\text{ctx}} = \Delta\theta_u^{(t)}. \quad (1)$$

**Parameter version.**

$$\Delta\theta_{u,\text{par}} = \text{TrainAdapter}(\Delta\theta_u^{(t)}, E_t), \quad M_{u,\text{par}} = M_u^{(t)}. \quad (2)$$

The two versions must use the same evidence, starting model, and later tasks  $V_t$ . The parameter version cannot receive  $E_t$  in its test prompt. Otherwise, the test cannot show what the adapter learned.

Use two selection conditions:

$$\begin{aligned} U_{\text{pref}}(m_{\text{par}}; V_t) &> U_{\text{pref}}(m_{\text{ctx}}; V_t), \\ U_{\text{task}}(m_{\text{par}}; V_t) &\geq U_{\text{task}}(m_{\text{ctx}}; V_t). \end{aligned} \quad (3)$$

Choose the parameter version only when both conditions hold. Otherwise, choose the context version. Do not update when the evidence is too limited or contradictory.

This rule belongs to the proposed experiment. It is not a finding from the papers above.

## 6 Experiment

### 6.1 Split the data by time

Order complete sessions before extracting profiles or creating training data:

$$H_u^{\text{past}} \prec E_t \prec V_t \prec H_u^{\text{test}}. \quad (4)$$

Use past records for initialization. Use  $E_t$  to build the two versions. Use  $V_t$  to select one. Keep the final period untouched for testing.

### 6.2 Minimum baselines

Report three outcomes separately:

- personalization on later tasks;
- task success verified by the environment; and
- prompt, training, and inference cost.

Do not combine them into one score. Use confidence intervals only to show how stable the results are.

Table 3: Baselines needed for the first experiment.

| Version                          | Question                                               |
|----------------------------------|--------------------------------------------------------|
| Base model                       | What happens without personalization?                  |
| Base model with prompt or memory | What can context do without training?                  |
| Adapter only                     | Did the parameters learn the user’s preferences?       |
| Adapter with prompt or memory    | Do the two forms help each other?                      |
| OPPU                             | Is standard per-user PEFT enough?                      |
| SPRInG                           | Is selective updating with retrieval enough?           |
| CoSteer, when practical          | Can runtime steering avoid training?                   |
| Proposed test                    | Does choosing where to store evidence improve results? |

## 7 Novelty Boundary

The following ideas already exist:

- one adapter for each user;
- continued updates to a user adapter;
- external memory used with an adapter; and
- comparisons between context and parameter learning.

The possible new point is narrower. Use the same evidence to build a context version and a parameter version. Then use the same user’s later tasks to decide where that evidence should stay.

This difference still needs a broader literature review and an experiment. For now, it is a research hypothesis.

## 8 Next Steps

1. Select same-user data with reliable timestamps.
2. Implement OPPU, SPRInG, and prompt or memory baselines from their papers or official code.
3. Fix one adapter training method and one memory update method.
4. Run the base, context-only, adapter-only, and combined versions.
5. Run the matched comparison proposed in this note.
6. Extend to tool-using agent tasks only after personalization and task success both hold.

## References

- [1] Zhaoxuan Tan, Qingkai Zeng, Yijun Tian, Zheyuan Liu, Bing Yin, and Meng Jiang. Democratizing Large Language Models via Personalized Parameter-Efficient Fine-Tuning. EMNLP, 2024. <https://aclanthology.org/2024.emnlp-main.372/>.
- [2] Xinyu Li, Ruiyang Zhou, Zachary C. Lipton, and Leqi Liu. Personalized Language Modeling from Personalized Human Feedback. arXiv:2402.05133, 2024. <https://arxiv.org/abs/2402.05133>.

- [3] Seoyeon Kim and Jaehyung Kim. SPRInG: Continual LLM Personalization via Selective Parametric Adaptation and Retrieval-Interpolated Generation. arXiv:2601.09974, 2026. <https://arxiv.org/abs/2601.09974>.
- [4] Rong Wu, Xiaoman Wang, Jianbiao Mei, and others. EvolveR: Self-Evolving LLM Agents through an Experience-Driven Lifecycle. arXiv:2510.16079, 2025. <https://arxiv.org/abs/2510.16079>.
- [5] Hang Lv, Sheng Liang, Hao Wang, and others. CoSteer: Collaborative Decoding-Time Personalization via Local Delta Steering. arXiv:2507.04756, 2025. <https://arxiv.org/abs/2507.04756>.